Experiment 10: L2, Dropout and Early Stopping Comparison

Question

Problem Statement: A medical-records team trained a large MLP that scores 99% on training data but performs poorly on new patients. A small tabular dataset (around 500 records) for predicting diabetes onset from clinical features. The objective is to compare L2 regularisation, dropout and early stopping as remedies for overfitting. Use TensorFlow, Keras, NumPy, Matplotlib.

Tasks:

1. Train a deliberately large baseline MLP and plot its overfitting.
2. Retrain with L2 regularisation at three λ values.
3. Retrain with dropout at rates 0.2, 0.3 and 0.5.
4. Retrain with early stopping using validation loss.
5. Tabulate test accuracy and train–test gap for every model.

Code

```
"""Experiment 10: L2, Dropout and Early Stopping Comparison"""
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, regularizers
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

np.random.seed(42); tf.random.set_seed(42)

# ---------------- Small tabular dataset (~500 records, 8 clinical features) ----------------
X, y = make_classification(n_samples=500, n_features=8, n_informative=4, n_redundant=2,
                           flip_y=0.1, class_sep=0.8, random_state=42)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.25, stratify=y, random_state=42)
sc = StandardScaler().fit(X_tr)
X_tr, X_te = sc.transform(X_tr), sc.transform(X_te)
EPOCHS = 200

def build(l2=0.0, dropout=0.0):
    reg = regularizers.l2(l2) if l2 > 0 else None
    m = keras.Sequential([layers.Input((8,))])
    for units in [256, 256, 128]:                       # deliberately large
        m.add(layers.Dense(units, activation="relu", kernel_regularizer=reg))
        if dropout > 0:
            m.add(layers.Dropout(dropout))
    m.add(layers.Dense(1, activation="sigmoid"))
    m.compile(optimizer=keras.optimizers.Adam(0.001), loss="binary_crossentropy", metrics=["accuracy"])
    return m

results, histories = [], {}
def run(name, l2=0.0, dropout=0.0, early=False):
    m = build(l2, dropout)
    cb = [keras.callbacks.EarlyStopping(monitor="val_loss", patience=15, restore_best_weights=True)] if early
else []
    h = m.fit(X_tr, y_tr, validation_split=0.2, epochs=EPOCHS, batch_size=32, callbacks=cb, verbose=0)
    tr_acc = m.evaluate(X_tr, y_tr, verbose=0)[1]
    te_acc = m.evaluate(X_te, y_te, verbose=0)[1]
    results.append({"Model": name, "Epochs run": len(h.history["loss"]), "Train acc": tr_acc,
                    "Test acc": te_acc, "Train-Test gap": tr_acc - te_acc})
    histories[name] = h.history
    print(f"{name:22} train={tr_acc:.3f} test={te_acc:.3f} gap={tr_acc - te_acc:.3f}")

# ---------------- Task 1: baseline ----------------
run("Baseline")
h = histories["Baseline"]
fig, ax = plt.subplots(1, 2, figsize=(12, 4))
ax[0].plot(h["loss"], label="train"); ax[0].plot(h["val_loss"], label="validation")
```

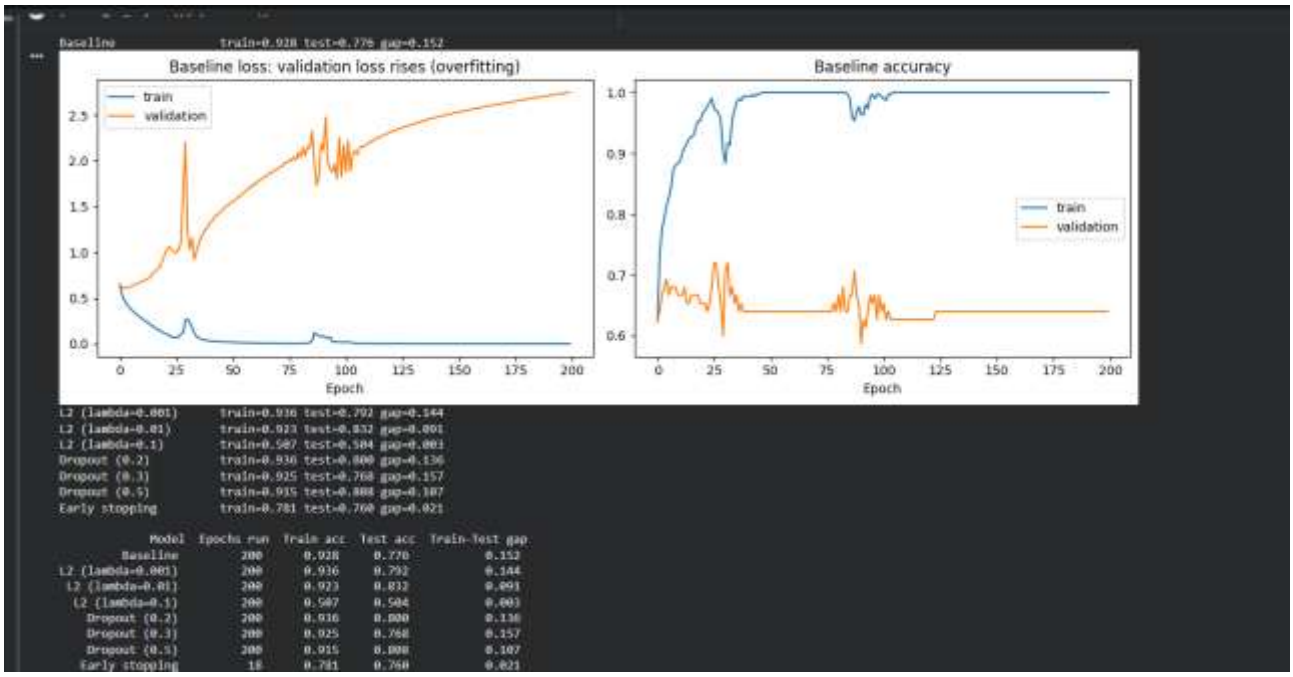
```
ax[0].set_title("Baseline loss: validation loss rises (overfitting)"); ax[0].set_xlabel("Epoch");
ax[0].legend()
ax[1].plot(h["accuracy"], label="train"); ax[1].plot(h["val_accuracy"], label="validation")
ax[1].set_title("Baseline accuracy"); ax[1].set_xlabel("Epoch"); ax[1].legend()
plt.tight_layout(); plt.show()


# ---------------- Task 2: L2 ----------------
for lam in [0.001, 0.01, 0.1]:
    run(f"L2 (lambda={lam})", l2=lam)
# ---------------- Task 3: Dropout ----------------
for r in [0.2, 0.3, 0.5]:
    run(f"Dropout ({r})", dropout=r)
# ---------------- Task 4: Early stopping ----------------
run("Early stopping", early=True)


# ---------------- Task 5: table ----------------
table = pd.DataFrame(results)
print("\n", table.round(3).to_string(index=False))


fig, ax = plt.subplots(1, 2, figsize=(14, 4.5))
for name in ["Baseline", "L2 (lambda=0.01)", "Dropout (0.3)", "Early stopping"]:
    ax[0].plot(histories[name]["val_loss"], label=name)
ax[0].set_title("Validation loss comparison"); ax[0].set_xlabel("Epoch"); ax[0].set_ylim(0, 2.5);
ax[0].legend()
ax[1].barh(table["Model"], table["Train-Test gap"]); ax[1].set_title("Train - Test accuracy gap (lower is
better)")
plt.tight_layout(); plt.show()
```

Output




```
Baseline            train=0.928 test=0.776 gap=0.152
L2 (lambda=0.001)   train=0.936 test=0.792 gap=0.144
L2 (lambda=0.01)    train=0.923 test=0.832 gap=0.091
L2 (lambda=0.1)     train=0.587 test=0.584 gap=0.003
Dropout (0.2)       train=0.936 test=0.800 gap=0.136
Dropout (0.3)       train=0.925 test=0.768 gap=0.157
Dropout (0.5)       train=0.915 test=0.808 gap=0.107
Early stopping      train=0.781 test=0.760 gap=0.021
```

Model	Epochs run	Train acc	Test acc	Train-Test gap
Baseline	200	0.928	0.776	0.152
L2 (lambda=0.001)	200	0.936	0.792	0.144
L2 (lambda=0.01)	200	0.923	0.832	0.091
L2 (lambda=0.1)	200	0.587	0.584	0.003
Dropout (0.2)	200	0.936	0.800	0.136
Dropout (0.3)	200	0.925	0.768	0.157
Dropout (0.5)	200	0.915	0.808	0.107
Early stopping	18	0.781	0.760	0.021

